- Treating a class as a partial double
- Using RSpec's stubbed constants
- Dependency injection, in its many forms

Each of these approaches has its advantages and trade-offs. As we look through them in turn, we'll be working with the same simple example: an APIRequestTracker class that helps API developers track simple usage statistics for each endpoint. This kind of information is handy for figuring out which features customers are engaging with the most.

For instance, in the expense tracker you built in *Building an App With RSpec 3*, you might want to count the following statistics:

- How often customers POST to /expenses, tracked as post_expense
- How often customers GET from /expenses/:date, tracked as get_expenses_on_date

Here's one way to implement APIRequestTracker:

15-using-test-doubles-effectively/10/api_request_tracker/lib/api_request_tracker.rb

```
class APIRequestTracker
  def process(request)
    endpoint_description = Endpoint.description_of(request)
    reporter = MetricsReporter.new
    reporter.increment("api.requests.#{endpoint_description}")
  end
end
```

First, we get a description of the endpoint (based on the path and whether it was a GET or POST) to use for tracking purposes. Next, we create a new instance of MetricsReporter, which will send statistics to a metrics service. Finally, we tell the reporter to increment the call count for this API endpoint. In our expense tracker example, we'd bump the api.requests.post_expense or api.requests.get_expenses_on_date metric.

The MetricsReporter collaborator is a good candidate for replacement with a mock object in our test. We'd like to run a unit spec without needing a network connection and a test account on a live metrics service.

We've got our work cut out for us if we want to use a test double with this class, though. It instantiates the reporter object from a hard-coded class name, with no easy way for a test to control which reporter gets used.

Expecting a Message on Any Instance

RSpec can get us out of difficult situations like this one, via its *any instance* feature. Instead of allowing or expecting a message on a specific instance of a class, you can do so on any of its instances: